Estadística en les Ciències de la Salut
Assignment 1; Group 03

1711342 Roger Guitart Casals; 1707055 Sergi Resina Xirgo
1705323 Lucca Bergonzini Flores; 1702697 Àlex Vega Tomàs

28/10/2026

1 Introduction

In this study we will investigate how to use R packages such as mice and miclust to apply statistical tools for missing data. Our goal is to generate clusters to analyze and visualize our dataset. This dataset is a subset of the variables studied in the original paper by Basagaña et al.[1]

2 Analysis

2.1 Description of missingness

```
n_participants <- nrow(copd)
n_variables <- ncol(copd)
missing_per_variable <- colMeans(is.na(copd)) * 100

min_missing <- sprintf("%.1f", min(missing_per_variable))
max_missing <- sprintf("%.1f", max(missing_per_variable))

pct_variables_lt5 <- sprintf("%.1f", mean(missing_per_variable < 5) * 100) # mean
# computes the proportion of TRUE values
# equivalent: AA <- sprintf("%.1f", sum((missing_per_variable < 5))/n_variables * 100)
# true=1, false=0

pct_complete_cases <- sprintf("%.1f", mean(complete.cases(copd)) * 100)
pct_missing_total <- sprintf("%.1f", mean(is.na(as.matrix(copd))) * 100)

total_cells <- prettyNum(n_participants * n_variables, big.mark=",") # n_participants
                                                    # * n_variables = total_cells
```

The subset used here included 342 participants and 45 variables. The range of missing data in the different variables went from 0.0% to 31.6%, with 68.9% of the variables having less than 5% of the data missing. Only 33.3% of the participants had complete data on all 45 variables. Overall, 5.4% of the information was missing; that is, 5.4% of the total of 15,390 cells (45 x 342 = 15,390) had missing data.

2.2 Multiple imputation

Apply multiple imputation to copd data using 100 imputations. Explain in a reasoned manner how you decided to set the values of the arguments method and predictorMatrix when using the mice() function. Show your code

```
ini <- mice(copd, maxit = 0)

meth <- ini$method
pred <- ini$predictorMatrix
```

```r
meth["id"] <- "" # do not impute "id"
pred["id", ] <- 0 # we don't want "id" to be imputed
pred[, "id"] <- 0 # we don't want "id" to be predictor

# Perform the 100 imputations:
imp <- mice(copd, method = meth, predictorMatrix = pred, m = 100,
            printFlag = FALSE, seed = 123)
```

We kept the same methodology and the predictor matrix, with the exception of id as it isn't relevant for this process. The variable 'm' encodes the number of imputations, 'printFlag' is used to show the current imputation although it's not important for the results, and the 'seed' is set up for replicability.

2.3 Integration of multiple imputation in cluster analysis

Reproduce the original analysis in the paper using the miclust package to integrate multiple imputation in the cluster analysis. Apply the proposed framework using the same setting as in the original analysis.

To do that, follow these steps:

1. Read the help of the getdata() function in the miclust package. Then, accordingly, create a list of data.frames and pass it to the getdata() function. Give a detailed explanation of the data processing you did to get that list. Show your code.

```r
resultat_mice <- complete(imp, action = "all", include = TRUE)
# action = "all" already puts the original data as the first element

categoriques <- c(
  "sin11_v1", "iam_chv1", "sin15_v1", "sin7_v1", "sin8_v1",
  "cccs1_v0", "vtit_catv1", "urg_ambv0", "sin5_v1",
  "sin6_v1", "graudisn_v1"
)

resultat_mice <- lapply(resultat_mice, function(d) {
  d$id <- NULL
  d[categoriques] <- lapply(d[categoriques], function(x)
    as.numeric(as.character(x)))
  d
})

miclustdata <- getdata(resultat_mice) # adapts the data to the miclust format
```

As 'getdata()' necessitates a list of dataframes, where the first element has the initial data and the imputations are stored in the subsequent elements, we call the 'complete' function that returns the needed information when using "action=all" and "include=TRUE". Furthermore, we must adapt the categorical variables to numerical values. Thus we define a function that transforms the values stored in text into numbers and using the 'lapply()' function, we reach our goal: the list of dataframes that 'getdata()' needs.

2. Use the miclust() function to perform the integration of missing data in cluster analysis using the same settings as in the original analysis. Show your code and explain how and why you set all arguments in the miclust() function.

```r
# Second part:
copd_miclustdata <- miclust(data = miclustdata,search = "backward",
                            ks = 2:3,
                            distance = "manhattan",
                            centpos = "means",
                            initcl = "hc",
                            verbose = TRUE,
                            seed = 123)
```

```
## . . . . imp   5 . . . . imp  10 . . . . imp  15 . . . . imp  20
## . . . . imp  25 . . . . imp  30 . . . . imp  35 . . . . imp  40
## . . . . imp  45 . . . . imp  50 . . . . imp  55 . . . . imp  60
## . . . . imp  65 . . . . imp  70 . . . . imp  75 . . . . imp  80
## . . . . imp  85 . . . . imp  90 . . . . imp  95 . . . . imp 100
##
##  Analysis done.
```

The chosen elements have been: 'backward', because we were instructed to during class; 'ks = 2:3', as the paper itself led us to believe that a high number of clusters would lead to inaccurate results; 'manhattan', the default option; 'means', as we are using the k-means algorithm; 'hc', we decided on a hierarchical initialization over a random one for better results; 'verbose = TRUE', to show the progress of this long process; 'seed=123' for replicability.

3 Results

Using plot() and summary() methods in the miclust package, provide plots$_1$ and tables for the results similar to those shown in the original paper. Also, show a barplot for the percentage of times each variable is selected among the imputed data sets, in decreasing order$_2$. Explain how you set all arguments in all functions and methods you use. Show your code.

```
plot(copd_miclustdata, metric="nclfreq", col.nclfreq = "blue")
```

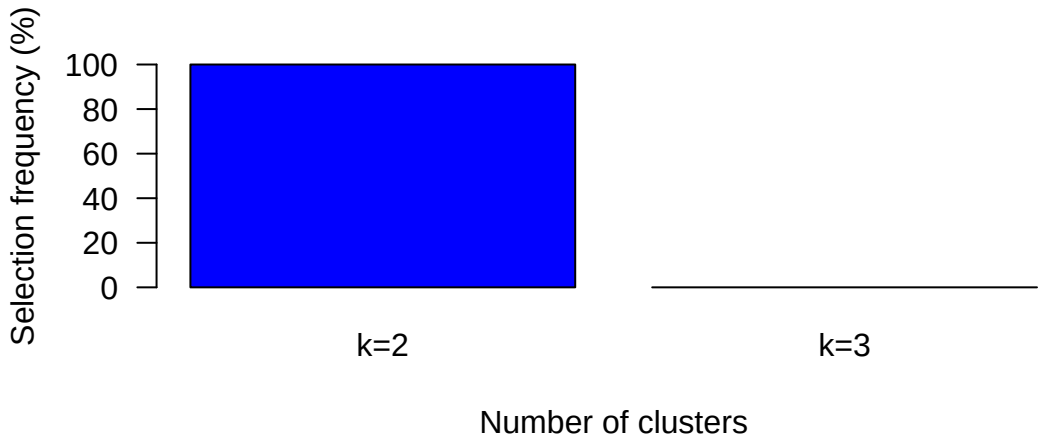


Figure 1: Number of clusters frequency

```
plot(copd_miclustdata, metric="critcf", col.critcf = "orange")
```

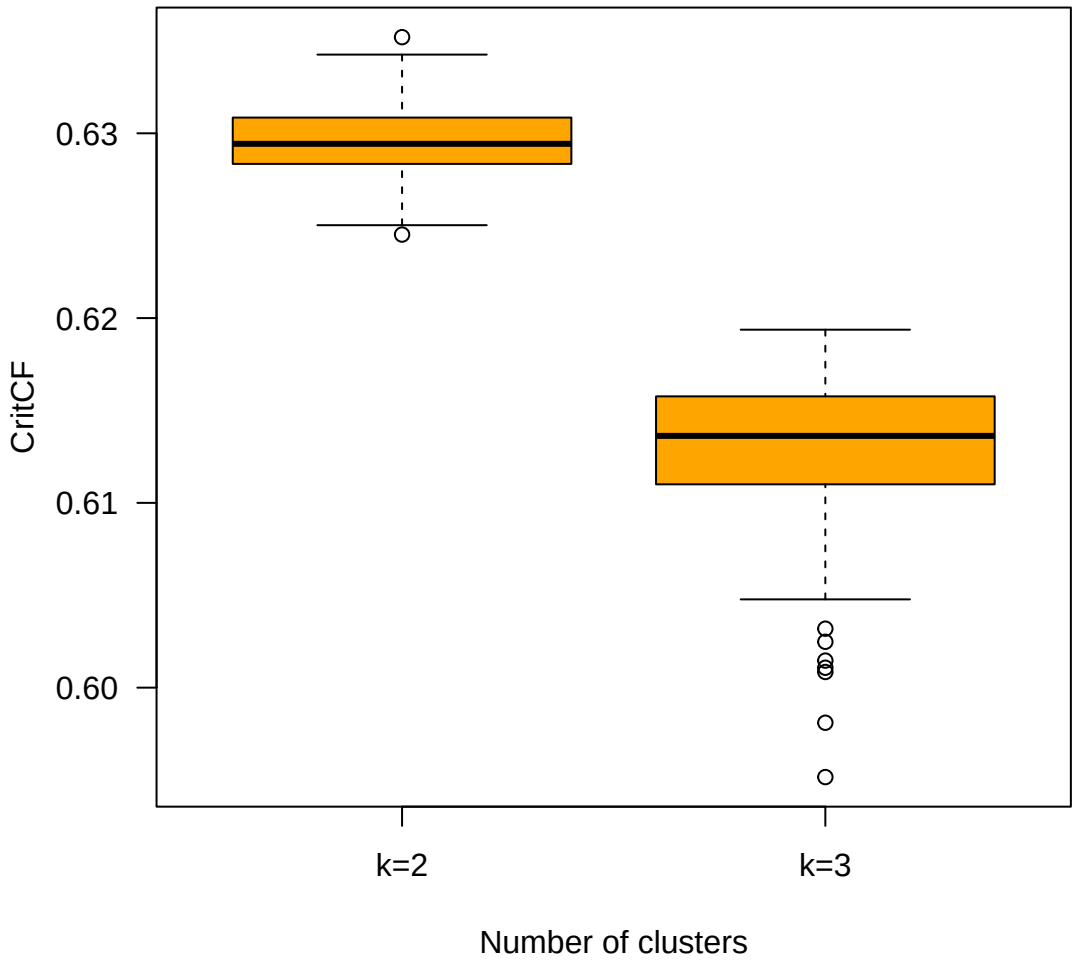


Figure 2: CritCF value versus number of clusters

```
plot(copd_miclustdata, metric="nvarfreq", col.nvarfreq = "forestgreen")
```

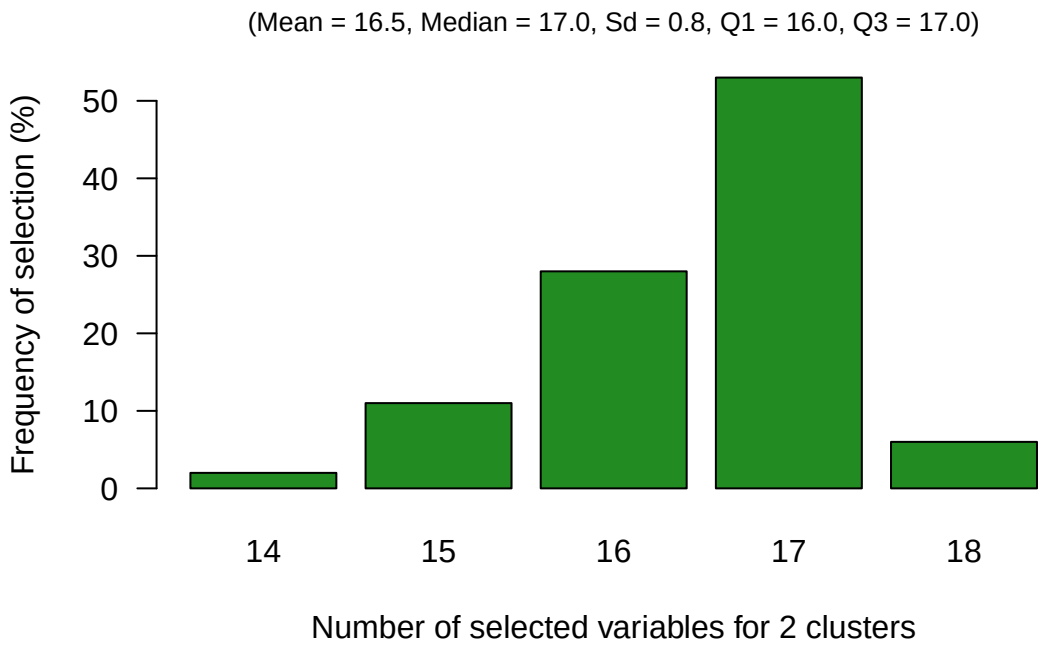


Figure 3: Frequency of the number of times each variable being selected furing multiple imputation

```
plot(copd_miclustdata, metric="varsel", col.varsel = "gray")
```

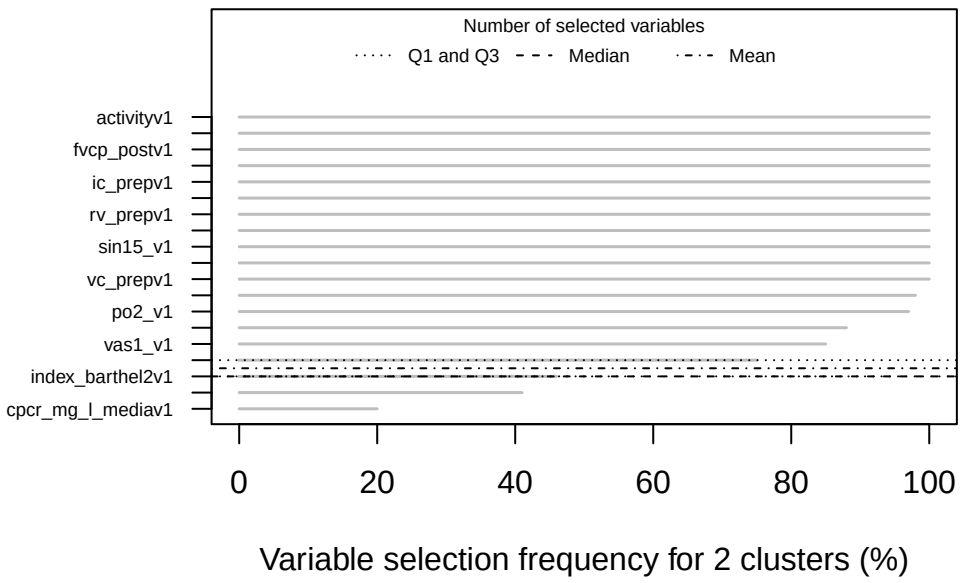


Figure 4: Frequency of each variable being selected during multiple imputation

We only needed to define the colors to show the plots.

```r
res2 <- summary(copd_miclustdata, k = 2, quantilevars = 0.5)
res3 <- summary(copd_miclustdata, k = 3, quantilevars = 0.5)

resum_prob <- rbind(res2$allocationprobabilities, res3$allocationprobabilities)

rownames(resum_prob) <- c("k = 2, Cluster 1",
                          "k = 2, Cluster 2",
                          "k = 3, Cluster 1",
                          "k = 3, Cluster 2",
                          "k = 3, Cluster 3")

modtable <- xtable(resum_prob,
                   digits = c(0, 3, 3, 3, 3, 3),
                   caption = "Distribution of the Frequencies of Assignment
                   (Proportions) for Each Cluster (for k = 2 and k = 3)",
                   label = "tab:frequency")
# cosmetics for table head (LaTeX slash (\) must be doubled inside a R chunk):
colnames(modtable) <- c("\\multicolumn{1}{c}{\\bf Minimum}",
                        "\\multicolumn{1}{c}{\\bf First Quartile}",
                        "\\multicolumn{1}{c}{\\bf Median}",
                        "\\multicolumn{1}{c}{\\bf Third Quartile}",
                        "\\multicolumn{1}{c}{\\bf Maximum}")

# format and print the table:
print(modtable,
      table.placement = "ht",
      size = "footnotesize",
      include.rownames = TRUE,
      include.colnames = TRUE,
      floating = TRUE,                  # whether \begin{table}
      # should be created (TRUE) or not (FALSE)
      # important to treat content
      # of columns as latex function
      sanitize.text.function = force)
```

	Minimum	First Quartile	Median	Third Quartile	Maximum
k = 2, Cluster 1	0.520	1.000	1.000	1.000	1.000
k = 2, Cluster 2	0.540	1.000	1.000	1.000	1.000
k = 3, Cluster 1	0.600	1.000	1.000	1.000	1.000
k = 3, Cluster 2	0.500	1.000	1.000	1.000	1.000
k = 3, Cluster 3	0.540	1.000	1.000	1.000	1.000

Table 1: Distribution of the Frequencies of Assignment (Proportions) for Each Cluster (for k = 2 and k = 3)

Answer: To create the first table we use the 'summary()' function for the different k-values, and using a template provided by the professor, we show the desired results.

```r
descr <- as.data.frame(res2$summarybycluster)

taula3 <- data.frame(Variable = gsub("_", "\\_", rownames(descr)),
                     Missing = as.numeric(descr[["%miss."]]),
                     Cluster1 = as.numeric(descr[["mean (cl.1)"]]),
                     Cluster2 = as.numeric(descr[["mean (cl.2)"]]),
                     check.names = FALSE
)

n <- res2$clusterssize[, "50%"] # Value of the median cluster size
```

```r
modtable <- xtable(taula3,
                   digits = c(0, 0, 2, 2, 2),
                   caption = "Description of the Variables (Mean Values) by Cluster",
                   label = "tab:descr")
# cosmetics for table head (LaTeX slash (\) must be doubled inside a R chunk):
colnames(modtable) <- c("\\multicolumn{1}{c}{\\bf Variable}",
                        "\\multicolumn{1}{c}{\\bf \\% of Subjects With Missing Values}",
                        paste0("\\multicolumn{1}{c}{\\bf Cluster 1 (n = ", n[1], ")}"),
                        # paste0 to concatenate strings
                        paste0("\\multicolumn{1}{c}{\\bf Cluster 2 (n = ", n[2], ")}"))

# format and print the table:
print(modtable,
      table.placement = "ht",
      size = "footnotesize",
      include.rownames = FALSE,
      include.colnames = TRUE,
      # whether \begin{table} should be created (TRUE) or not (FALSE)
      floating = TRUE,
      # important to treat content of columns as latex function
      sanitize.text.function = force)
```

Variable	% of Subjects With Missing Values	Cluster 1 (n = 186)	Cluster 2 (n = 156)
activityv1	1.17	35.80	60.79
fevp_postv1	0.00	62.74	39.12
fvcp_postv1	0.00	79.93	63.65
iam_chv1	0.88	0.12	0.08
ic_prepv1	5.56	72.84	52.43
quo_calcpostv1	0.00	58.83	45.89
rv_prepv1	7.89	138.51	184.77
rv_tlcv1	7.89	50.89	63.19
sin15_v1	1.17	0.09	0.09
vas2_v1	1.17	4.18	6.49
vc_prepv1	5.56	77.48	60.45
impactsv1	1.17	18.61	35.81
po2_v1	3.22	77.39	69.67
graudisn_v1	1.17	1.91	3.27
vas1_v1	1.17	2.00	3.98
cccs1_v0	1.46	0.15	0.18
index_barthel2v1	2.63	100.00	98.15

Table 2: Description of the Variables (Mean Values) by Cluster

We haven't included the 'F values' in the second table, nor used the unprocessed data for the average-value analysis, but we have decided to show other results from the 'summary()' function, which we considered relevant to the imputed data. To do this, an auxiliary dataframe of the 'summary' was needed. Furthermore, we obtained the mean value of the clusters just like the original paper did.

4 Report

The final analysis divided the dataset into **2 clusters**. We can see in Figure 1 that k=2 (having 2 clusters) is chosen every time, thus we can conclude without uncertainty that we can divide the dataset into two clusters. Table 2 details the size of each of these clusters: n=186 for the first cluster and n=156 for the second one.

The relabeling of the clusters over multiple imputations (the same cluster being labelled differently in different imputations) is handled internally by the **miclust** package, as detailed in its documentation. [2]

Figure 3 shows how many variables were selected across the imputations: an average of 16.5, with a median of 17, standard deviation of 0.8 and an interquartile range of 16-17. Thus, the final subset contains 17 variables and the uncertainty over this choice is low. Figure 4 shows us the selection frequency for each variable: the most selected include activityv1, fvcp_postv1, ic_prepv1, rv_prepv1, sin15_v1,

vc_prepv1, po2_v1, vas1_v1, index_barthel2v1 and cpcr_mg_l_mediav1. Table 2 lists the variables used in the descriptive cluster.

In Table 1, we can see that, for each cluster when k = 2 and k = 3, at least three-quarters of its instances are assigned to the same cluster across all imputations. Therefore, assuming that the missing data follow the proposed distributions, at least 75% of all instances don't have uncertainty.

5 Use of artificial intelligence

Artificial Intelligence, more accurately Large Language Models (LLMs), have been used to revise and enhance the code displayed in this paper. Here is the exact prompt and answer of the one time they were used:

Prompt:

```
Tinc aquest codi en R:
resultat_mice<-complete(imp, action="all", include=TRUE)
categorics_amb_ordre <- c("sin11_v1", "iam_chv1", "sin15_v1", "sin7_v1",
"sin8_v1","cccs1_v0", "vtit_catv1", "urg_ambv0", "sin5_v1", "sin6_v1")
categorics_sense_ordre <- c("graudisn_v1")
for (i in 1:(M+1))
{
  for (j in categorics_amb_ordre)
  {
    resultat_mice[[i]][[j]] <- as.numeric(levels(resultat_mice[[i]][[j]]))
    [resultat_mice[[i]][[j]]]
  }
  for (j in categorics_sense_ordre)
  {
    resultat_mice[[i]][[j]] <- as.numeric(levels(resultat_mice[[i]][[j]]))
    [resultat_mice[[i]][[j]]]
  }
}
miclustdata<-getdata(resultat_mice)


Saps si es pot optimitzar i simplificar? També voldria eliminar la columna
id de cada conjunt de dades imputat abans de crear miclustdata.
```

Answer:

```
Sí. En el teu codi original hi ha dues simplificacions clares:

1. categorics_amb_ordre i categorics_sense_ordre reben exactament el mateix tractament,
   per tant es poden unir en un únic vector.
2. Es poden evitar els for utilitzant lapply(), cosa que fa el codi
   més compacte i independent de M.
3. Pots eliminar la columna id dins del mateix procés.

Una possible versió simplificada seria:


resultat_mice <- complete(imp, action = "all", include = TRUE)


categoriques <- c(
  "sin11_v1", "iam_chv1", "sin15_v1", "sin7_v1", "sin8_v1",
  "cccs1_v0", "vtit_catv1", "urg_ambv0", "sin5_v1",
  "sin6_v1", "graudisn_v1"
)
```

```
resultat_mice <- lapply(resultat_mice, function(d) {

  # Eliminar la variable identificadora
  d$id <- NULL

  # Convertir factors a numèrics
  d[categoriques] <- lapply(
    d[categoriques],
    function(x) as.numeric(as.character(x))
  )

  d
})

miclustdata <- getdata(resultat_mice)


Aquesta versió manté la mateixa funcionalitat sempre que les
variables categòriques tinguin nivells numèrics (per exemple "0", "1", "2", ...),
elimina la variable id i evita haver de controlar explícitament el nombre
d'imputacions M.
```

Model and Version: It was generated using M365 Copilot, which is powered by ChatGPT-5

Reflection: This project has been aided by LLMs in a specific instance of code checking and improvement. Our use of AI has been contained and minor, as, in an academic environment where our goal is to learn, we consider the use of these tools counter-productive. Thus, our final product and the process of creating it have not been extensively influenced by AI.

References

[1] X. Basagaña, J. Barrera-Gómez, M. Benet, JM. Antó, and J. Garcia-Aymerich. *A Framework for Multiple Imputation in Cluster Analysis.* American Journal of Epidemiology, 177(7):718–725, 2013. URL https://doi.org/10.1093/aje/kws289.

[2] J. Barrera-Gomez and X. Basagana. *miclust: Cluster Analysis in Multiple Imputed Datasets with Optional Variable Selection.* R package version 1.2.8, 2022. URL https://rdrr.io/cran/miclust/man/miclust.html.